

Machine Learning based Automated Resume Shortlisting System

A PROJECT REPORT

Submitted by

David. R (180071601072)

Mohamed Imran (180071601108)

Under the guidance of

Dr. NIYATI KUMARI BEHERA

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

**B.S.ABDUR RAHMAN CRESCENT INSTITUTE OF
SCIENCE & TECHNOLOGY**

(Estd. u/s 3 of the UGC Act. 1956)

<https://crescent.education/>



JUNE 2022

**B.S.ABDUR RAHMAN CRESCENT INSTITUTE OF
SCIENCE & TECHNOLOGY**

(Estd. u/s 3 of the UGC Act. 1956)

<https://crescent.education/>



BONAFIDE CERTIFICATE

Certified that this project report “MACHINE LEARNING BASED AUTOMATED RESUME SHORTLISTING SYSTEM” is the bonafide work of “David. R (180071601072) and Mohamed Imran. M (180071601108)” who carried out the project work under my supervision. Certified further, that to the best of our knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. NIYATI KUMARI BEHERA

Assistant Professor

Department of CSE

B.S. Abdur Rahman Crescent

Institute of Science and Technology

Vandalur, Chennai – 600048

SIGNATURE

Dr.E. SYED MOHAMED

Head of the Department

Professor & Head

B.S. Abdur Rahman Crescent

Institute of Science and Technology

Vandalur, Chennai – 600048

**B.S.ABDUR RAHMAN CRESCENT INSTITUTE OF
SCIENCE & TECHNOLOGY**

(Estd. u/s 3 of the UGC Act. 1956)
<https://crescent.education/>



VIVA VOCE EXAMINATION

The viva voce examination of the project work titled “**MACHINE LEARNING BASED
AUTOMATED RESUME SHORTLISTING SYSTEM**”, submitted by **David. R
(180071601072)** and **Mohamed Imran. M (180071601108)** is held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We sincerely express our heartfelt gratitude to **Dr. A. PEER MOHAMED**, Vice Chancellor, B.S. Abdur Rahman Crescent Institute of Science and Technology, for providing us an environment to carry out our course successfully.

We sincerely thank **Dr. N. RAJA HUSSIAN**, Registrar for furnishing every essential facility for doing our project.

We thank **Dr. VENKATESAN SELVAM**, Dean, School of Computer, Information and Mathematical Sciences for his motivation and support.

We thank **Dr. E. SYED MOHAMED**, Professor and Head, Department of Computer Science and Engineering, for providing strong oversight of vision, strategic direction and valuable suggestions.

We express our sincere thanks to the Project Review Committee members, from the Department of Computer Science and Engineering, **Dr. S.REVATHI**, Professor, and **Dr. X. ARPUTHA RATHINA**, Associate Professor for their valuable suggestions and support.

We obliged supervisor **Dr. NIYATI KUMARI BEHERA**, Assistant Professor, Department of Computer Science and Engineering for his professional guidance and continued assistance during our project.

We thank our class advisor, **MRS. C. VIJAYALKSHMI**, Assistant Professor (Sr.gr), Department of Computer Science and Engineering for her guidance and encouragement throughout the project period.

We thank all the **Faculty members** and the **System Staff** of the Department of Computer Science and Engineering for their valuable support and assistance at various stages of project development.

(DAVID. R)

(MOHAMED IMRAN. M)

Abstract

Recruitment of pertinent people for certain positions is a crucial task for a company or individual organizations. Manual screening for selecting the appropriate candidate from a large amounts of resumes can be exhausting and time consuming. The final selection of the qualified candidate might be less efficient to the required job description of the company due to human errors. It may miss the right applicants or select unsuitable applicants for the job. A lot of work has been done for the job search process. The process of selecting a suitable applicants is based on applicant's resume and ranking system is not fully automated. Although there had been a few types of research to automate the system in some different ways and there were some studies to make the process much less uninteresting and less difficult at similar instances, there are still a few rooms for improvement. The way of automated resume classification and matching the job description make simpler the tough process of fair shortlisting. An approach of mixing NLP with ML seems like a viable solution for Resume Classification. The major motive of this system is to take the current resume ranking system to another level and makes it more flexible for both the entity. To propose a system that provides a list of applicants with appropriate experience and then represents the value earned of each selected resume, unlike the conventional way of applying filters and manually scanning resumes. To identify the most suitable candidates for a particular vacancy, a recruiter can easily select qualified candidates in a short time where the qualified candidates are ordered based on a ranking system and where the applicants can also demonstrate their creativity while maintaining just one format to apply in different organizations. The proposed system produce the good results. This process is also not time consuming.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	v
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
	LIST OF TABLES	x
1	INTRODUCTION	1
1.1	OVERVIEW	1
1.2	DESCRIPTION	2
1.3	OBJECTIVE	3
2	LITERATURE SURVEY	4
3	PROBLEMS DEFINITION AND METHODOLOGIES	9
3.1	PROBLEMS DEFINITION	9
3.2	EXISTING SYSTEM	9
3.3	PROPOSED SYSTEM	10
3.4	LDA TOPIC MODELLING	11
3.5	METHODOLOGY	11
3.5.1	NATURAL LANGUAGE PROCESSING	11
3.6	TF-IDF	13
3.7	COSINE SIMILARITY	14
3.8	OVERLAP COEFFICIENT SIMILARITY	14
3.9	JACCARD SIMILARITY	15
3.10	DICE SIMILARITY	15
4	DESIGN PROCESS	16
4.1	SYSTEM DESIGN	16
4.2	SYSTEM REQUIREMENTS	18
4.2.1	Hardware requirements	18
4.2.1	Software requirements	18
4.3	MODULES	18

5	IMPLEMENTATION	21
5.1	DROP DOWN FOR DISPLAYING NAMES	21
5.2	DROP DOWN FOR DISPLAYING CONTENT	21
5.3	TEXT PRE-PROCESSING	22
5.4	TF-IDF CALCULATION	22
5.5	RANKING EACH CANDIDATES THROUGH SCORES	23
6	CONCLUSION AND FUTURE ENHANCEMENTS	26
	References	27
	Appendix	29
	A1-Source code	29
	A2-Screenshots	44
	A3-Similarity Index Report	48
	Technical Biography	49

LIST OF FIGURES

FIGURE.NO	TITLE	PAGE.NO
4.1	System architecture	16
5.1	Job description name list	21
5.2	Job description content	22
5.3	Bar chart	24
5.4	Word cloud	24
5.5	Topic Modelling	25
8.1	No. of. Job description	44
8.2	Job description selection	44
8.3	Job description content	45
8.4	Ranking table	45
8.5	Bar char for ranking	46
8.6	Word cloud	46
8.7	Topic modelling for individual	47

LIST OF ABBREVIATIONS

S.NO	ABBREVIATIONS	DESCRIPTION
1	IT	Information Technology
2	NLP	Natural Language Processing
3	ML	Machine Learning
4	AI	Artificial Intelligence
5	TF-IDF	Term Frequency – Inverse Document Frequency
6	LDA	Latenent Dirchlet Allocation
7	EDA	Exploratory Data Analysis
8	JD	Job Description
9	HR	Human Resources
10	NLTK	Natural Language ToolKit
11	GUI	Graphical User Interface

LIST OF TABLES

TABLE.NO	TITLE	PAGE.NO
4.1	System requirements	18
5.1	Score Table	23

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The progressing technology generating many probabilities of employment. Resume is the most important document to apply for any job nowadays. A curriculum vitae tells a lot about the person's accomplishments and the skill sets in all walks of life. The person run in for the work, highlights the solid points and skill sets required for the corporation. World-wide administrations receive thousands of emails from such individuals who send their resumes for them to apply for a certain post. The real contest now is to figure out which resume should be dealt with and shortlisted with respect to the requirements. One way is through manual checking and sorting each resume. Now this technique is the most time-consuming and also can tip to a lot of mistakes because of human involvements. Also, humans cannot keep on working continuously. There is also the difficult of low concert.

Proposed system that will easily find the required skill set by looking over the document or the resume and category allowing to the skill sets which is a specified constraint of the organization. Day by day many new explorations are being carried out by many corporations in a lot of fields. Right from the IT sector to the Medical fields, there are so many researches as well are progress made in day to day life.

Progress in these fields are creating new employment opportunities all over the world. The hiring process is known all over the world are same, meaning the applicant has to create the application first and most of the recruitment processes are based on the applicant's resume. At present, most of the companies ask applicants to send the resume by email. As a result, recruiters receive the email of the applicants and each document of the applicant is evaluated by the recruiters according to their requirement.

In a traditional way, recruiters go through each resume manually but this progression takes a lot of time and the recruiters might select an inappropriate candidate for the job due to human errors. The competence of such categorization is enormously low and this may also result in missing the candidate who is a suitable for the organization.

1.2 DESCRIPTION

Enrolment deals with hiring the best suitable nominees having the applicable skills for a given job profile from a massively large lake of candidates. If there is any job opening for a place in a company, a large aggregate of candidates send their curriculum vitae to the company to apply for that position. In the appointment process, the first task of the recruiter is to screen the bio data of entire job applicants. Any corporation that has a job proposal for a particular position will receive thousands of emails daily from job candidates. Choosing the prospective candidates for that job position from a large lake of aspirants for any recruiter is very tedious. It is an enormously formidable task for the recruiters of a company to physically go through thousands of resumes and select the most fitting candidates for the job.

Due to this, the recruiters rather over and over again find it really difficult to narrow down the most appropriate candidates from a large aspirant lake. In latest years, nearby more than 60,000 e-recruitment sites have been established. The developers of these online recruitment sites have used various approaches to identify the prospective candidates for a given job profile of a company. Some of these, have achieved to employ classification techniques that will categorize the candidate curriculum vitae into several categories for every job spot given by every company. In these tactics, every candidate's curriculum vitae is tried to match with every given job posting on the enrolment site. The aim of these recruitment sites is to pitch up the outcomes to the aspirant to which are the greatest fit.

The creation of AI and ML has grown expressively in recent times. The elaboration of use of large volumes of data procured about by improvements in technology which has made the internet cut-rate and reachable to previously difficult to get to sections of the world has contributed to an excessive in the act of the ML models in current periods. Software companies around the world exploit the advances in ML to drive automation and increase their productivity in areas that relied mostly on manual human labor. The method discussed in this project is projected to routine the section-based division for data extraction using Natural language Processing. In order to keep up the time efficiency of the web application, the individual's CV will only be matched to those job openings where they are attentive in and have applied to which will, in turn, reduce the time complexity.

In addition, the results of the CV matching of all the nominees who have applied for the job opening will be visible only to the recruiter of that actual company. This is done with the aim to aid the recruiters of any corporation from the long and tedious task of observing and studying thousands of candidates' curriculum vitae.

1.3 OBJECTIVE

The proposed shortlisting system tries to improve the time efficiency and accuracy of the present curriculum vitae ranking system. To propose a system that provides a list of applicants with appropriate experience and then represents the value earned of each selected resume, unlike the conventional way of run in filters and physically scanning curriculum vitae. To recognise the most appropriate applicants for a particular position. It could be used in MNCs where multiple resumes must be screened every single day for multiple jobs, government, and administrative offices. There are quite a lot of attempts to programme several phases of the recruitment process. The objective of this curriculum vitae screening is to find the best applicant for a position. In order to tie and rate nominees in real-time, the system must employ NLP and similarities. This projected system of curriculum vitae ranking software uses NLP and similarities. With a set of individual candidates resumes and job descriptions for required domains of a company is integrated as input for the proposed system, while the results portraits as ranking based discrete scores of each candidates resume for the selected. The output is acquired with improved time efficiency. Scores set to the each applicant's and applicant's name shows in table that ordered from highest match to lowest match.

Output provides a ranking after filtering and ranks up the better resume for a given textual job description. Word tokenization, Data cleaning, Lemmatization, TF-IDF, Jaccard similarity, Dice similarity, Cosine Similarity, Overlap similarity such algorithms helps to achieved the excepted output. The extraction of tokens and related parameters matches the applicant's resumes and the job descriptions. Proposing a web application that aims to order the resumes, by intelligently reading job descriptions as input and comparing the resumes which fall into the category of given Job Descriptions. In order to competition and rate applicants in real-time, the software works NLP It make available a ranking after filtering and vouch for the better curriculum vitae for a given textual job description.

CHAPTER 2

LITERATURE SURVEY

Atharva Kulkarni et al. [1] proposed a system to examine different machine learning approaches for efficiently predicting personality through CV analysis using Natural Language Processing (NLP) techniques. The authors have used various Machine Learning Algorithms such as Logistic Regression, Naive Bayes, Random Forest, SVM and KNN for Personality prediction using CV Analysis. The authors have also used pyresparser, spaCy and PhraseMatcher and were able to predict the personalities of various candidates. The results indicated that Random Forest has the maximum accuracy of 0.71. Nevertheless, the authors concluded that due to lack of available data, the accuracy is much lesser than it was anticipated.

Md Tanzim Reza et al. [2] proposed a model for extracting important information from the semi-structured text format in a curriculum vitae or resume and ranking it according to the preference of the associated company and requirements. The structured data extraction process is done by segmenting the entire CV / Resume by converting it to HTML. After the conversion to structured data, decision tree algorithm techniques are used to classify the input into different categories based on qualifications and then the data with positive weight is used to train the system for future benefit. The classifier algorithm apart from decision tree such as logistic regression is used to compare the classification result. The proposed model extracts necessary data from a CV/Resume and segments them based on their values. The whole process was segmented and each segment was designed separately to perform its task. The segment that deals with Natural Language Processing actually worked with only the Natural Language Processing task. Similarly the segments that deal with Machine Learning, completely deals with Machine Learning techniques.

Sujit Amin et al. [3] proposed a web application designed in such a way that job applicant as well as recruiters can use it with ease for applying for job openings and screening respectively. Recruiters from various companies can post the details of the job openings available in their respective companies. This interactive web application allows the job applicants to submit their resume and apply for job postings. The resumes submitted by the candidates are then compared with the job profile

requirement posted by the company recruiter by using techniques like machine learning and Natural Language Processing (NLP). The proposed system used semi-supervised learning to achieve a higher accuracy. This system aided the recruiters to filter out the most prospective candidates based on their resumes for further rounds in the hiring process. The proposed system uses semi-supervised learning such as NER, NLTK to achieve a higher accuracy.

Jitendra Purohit et al. [4] developed an interviewing chatbot designed for companies to make recruitment process simpler and convenient. JARO accelerates the interview process towards an unbiased decision-making process by proposing a chatbot that would conduct interviews by analysing the candidates CV, and then prepares a set of questions to be asked to the candidate. The system consisted of features like resume analysis and automatic interview processes. The JARO Chatbot itself learns from all the interviews it conducts and further decides questions for the candidates. JARO examined the candidate on the areas of interest of candidate matching with job profile the company is looking for.

Tejaswini K et al. [5] proposed a system which will be able to accept or reject a job applicant based on two factors: the company's requirements must match the skills listed in the applicant's resume, and the test evaluation will be based on the applicant's skills, ensuring that the resumes uploaded by the applicant are genuine and the applicant is truly knowledgeable about the skills. The proposed model uses cosine similarity to find the curriculum vitae that are the most comparable to the job description supplied and the KNN algorithm is used to pick and rank Curriculum Vitae (CV) based on job descriptions in huge quantities. NLP methods and KNN algorithm were applied to rate resumes, assisting the firm in hiring the most qualified people.

Pradeep Kumar Roy et al. [6] proposed an automated machine learning based model which recommends suitable candidate's resume to the HR, based on given job description. The proposed model worked in two phases first, classify the resume into different categories and Recommended resume based on the similarity index with the given job description. The proposed approach effectively captures the resume sights, their semantics and yielded an accuracy of 78.53% with Linear SVM classifier. Top candidates could be ranked using Content-based Recommendation, using cosine similarity and using k-NN to identify the CVs that are nearest to the provided job

description. The performance of the model may enhance by utilizing the deep learning models like Convolutional Neural Network. Recurrent Neural Network, or Long Short Term Memory.

Abeer Zaroor et al. [7] addressed the issues associated with the previously highlighted techniques, been presented in a hybrid approach that employs conceptual-based classification of resumes and job postings and automatically ranks candidate resumes (that fall under each occupational category) to their corresponding job postings.

Irfan Ali et al. [8] proposed a methodology which uses several NLP and ML techniques for pre-processing data, feature extraction and representation, model construction, and evaluation for the Resume Classification task. Study also measures the accuracy based on finding k-nearest data points to the new instance and assign the label according to the highest neighbouring data points. This study results suggested that the TF-IDF vectorizer performed best in feature extraction and representation as the extracted features yielded excellent results on almost all classifiers. The study results were quite encouraging to automate the job application categorization and recommendation based on the content of the Resumes.

Anushka Sharma et al. [9] focuses on extracting data from resumes and performing the required analysis on the data to convert it into useful information for the recruiters. Resume Parser helped the recruiters to select the best relevant candidates in a minimal amount of time, consequently saving their time and effort.

Frank Farber et al. [10] proposed an approach to improve the quality of matching candidate profiles with job requirements based on recent research on automated recommendation. The approach could be used for any combination of input data and target attribute to be assessed. During a selection process, using several sources of candidate data and consisting of several levels of clinical as well as mechanical data combination, the method might be useful at several points. However, it was probably not capable of automating larger parts of the process. It is rather intended for instruments that are already based on electronic platforms, namely search in online resume databases and filtering processes in corporate applicant databases. The method developed uses a probabilistic latent aspect model that is able to capture a combination of different factors that might lead to a certain assessment. Using a small set of synthetic candidate profiles, the authors was able to show the feasibility with

first test runs. The estimated models successfully predicted the aptitude of candidates for a specific job profile.

V. V. Dixit et al. [11] proposed a project sorting all the resumes according to the requirement of the company and forwarded it to the respective HR for further recruitment process. The developer used the technology of Artificial Intelligence. The developer trained the system for recognizing the words by coding in Python language. The system was trained in such a way, when a resume is to be scanned, it will search only for the words or the special skills according to the requirement by the company and sort it accordingly. The required resume is shortlisted and the rest are rejected. This will give high efficiency compared to the manual sorting and give produce good results.

Ankit Basarkar et al. [12] Ankit Bhaskar et al.[12] implemented a project Document Classification through Machine Learning and concluded that inspite of binary vectorizing and the count vectorizing worked well, it is the Tf-Idf vectorizer that outperforms the two in terms of both appropriate representations of documents and the classification results. Although count vectorizer performed well and better than binary vectorizer, it performs poorly than binary vectorizer, if stop words are not removed. The developer used just one algorithm, and that is Naïve Bayes for classification as the main aim of the project was to analyze the different types of feature representation for documents. The project aims at comparing the Binary, Count and Tfidf feature vectors and their impact on document classification. To test how well each of the three mentioned feature vectors perform, developing group used the 20-newsgroup dataset and converted the documents to all the three feature vectors.

Vishnu M Menon et al. [13] proposed a system that processes the technical eligibility criteria based on the entries made by the users through online resumes and the applicants' emotional aptitude by leveraging their presence in the social media, which has become a huge repository of the collective interests and opinions of its users, who are motivated to share information and receive feedback from friends and colleagues. The Big Five personality model was used to perform the linguistic analysis. The approach was tested using data from three different datasets available in the literature and presented an average accuracy level of around 87% when using the algorithm

independent approach, while the algorithm dependent approach showed a vastly inferior average accuracy of about 45%. Employee grading was carried out using supervised machine learning algorithms that are trained with previous recruitment data. To estimate the performance, the system was implemented as a web application. The grading algorithms were trained using employee records composed from a corpus of 100 applicants with a recruiter assigning suitable eligibility scores. The trained system was found to be effective in predicting the scores and ranking users accordingly. An average positive correlation of around 65% between the scores assigned by the recruiter and those predicted by the system was observed when the algorithms were evaluated using Pearson correlation coefficient.

Rohini Nimbekar et al. [14] proposes a model of extracting valuable information from the resume and ranking it according to the preference and requirement of the company.

Allan Robey et al. [15] proposed a system which enables a more effective way to short list submitted candidate CVs from a large number of applicants providing a consistent and fair CV ranking policy. System would rank the experience and key skills required for a particular job position than system would rank the CV's based on the experience and other key skills which are required for particular job profile. The proposed system will help the HR department to easily shortlist the candidate based on the CV ranking policy. The authors have implemented an organization – oriented recruitment system which assist the HR department in short listing the right candidate for a specific job profile. Candidate gave an online test, which will be conducted on personality questions as well as aptitude questions. After completing the online test, candidate can view their own test results in graphical representation with marks. The system would then derive and rank the candidates who were eligible for the job. The rank of each candidate acts as a score of how well the candidate's profile meets the specifications of the recruiters as well as cumulative score of the aptitude test. The system could be used in many business sectors that will require expert candidate, thus reducing the work load on the human resource department. It presented a set of techniques that makes the whole recruitment process more effective and efficient.

CHAPTER 3

PROBLEM DEFINITION AND METHODOLOGIES

3.1. PROBLEM DEFINITION

Independent companies / organizations are in the process of recruiting talented people through job fairs from colleges. The process of assigning projects to new hires is usually a manual affair carried out by the organization's human resources department. This project allocation process for new recruits is a costly affair for the company as it largely relies on human effort. In latest times, software corporations everywhere the world are leveraging the improvements in machine learning and Artificial intelligence, in over-all, to systematise routine tasks in the enterprise to increase productivity. These recruitments are made on the basis of the skill set mentioned in the applicant's bio data. Now nearby a more of people who apply for a job. Physically observing the bio-data of these persons is more time consuming and less effective as it is inclined to inaccuracy in human intervention. The proposed system screens to quickly select candidates that meet specified requirements via the user interface. Thus, system able to achieve a more uniform screening process, lower cost to the organization, and identify better candidates

3.2. EXISTING SYSTEM

- In the first generation hiring model, the companies would advertise their vacancies in newspapers and on television. Applicants sent their resumes via post and their resumes would be sorted manually. Once shortlisted, the hiring team would call the applicants for further rounds of interview. Needless to say, this was a time-consuming procedure.
- The industries started growing and so did the hiring needs. Hence the companies started outsourcing their hiring process. Hiring consultancies came into existence. These agencies required the applicants to upload their resumes on their websites in particular formats. The agencies would then go through the structured data and shortlist candidates for the company. This process had a major drawback. There were numerous agencies and each had their own unique format.

- Organizations tries to builds the automated way to shortlisting the resumes. The IT industries finally completed the model with some limit such as single domain shortlisting.
- For automated shortlisting system in existence has poor user interface.

3.3. PROPOSED SYSTEM

The proposed system will allow the job applicants to submit resumes and apply for job postings with respect to the given job description. The application aims to align the applicants resume, by reading the selected job description which is given as input and comparing the resumes which fall into the category of the given Job Descriptions.

It provides a ranking after filtering and recommends the better resume for a given textual job description. This site utilizes Word Tokenization, Identifying stop words, Predicting Parts of Speech, Lemmatization, jaccard similarity, dice similarity, overlap similarity, and cosine similarity algorithms.

The effort takes a different method as it concentrations mainly on the content of the curriculum vitae where will achieve the extraction of skills and related parameters to match applicants with the job descriptions.

The curriculum vitae submitted by the applicants are then related with the job profile condition posted by the company. Scores set to the curriculum vitae and can be ordered from highest match to lowest match.

This ranking is made noticeable only to the company recruiter who is interested to choice the best applicants from a large lake of candidates.

While going through all these pipelines, ranking system generate score for each resumes and give. Resumes from the Data set are parsed to remove white spaces, numbers, stop words like and, or, etc. TF-IDF vectorization is then applied to convert the words in the resumes to vectors. The text in the job description is also converted to vectors using the TF-IDF vectorizer. Cosine, jaccard, dice, overlap distance is computed to measure the similarity between the resume and the job description provided

3.4 LDA topic modelling

Latent Dirichlet allocation is one of the most popular methods for performing topic analysing. Each document consists of various words and each topic can be associated with some words. The aim behind the LDA to find topics that the document belongs to, on the basis of words contains in it. It assumes that documents with similar topics will use a similar group of words. This enables the documents to map the probability distribution over latent topics and topics are probability distribution.

Using the LDA algorithm, through unsupervised classification methodology called Topic Modelling is implemented. With automated analysis of each resumes, utilizing word clouds to highlight each topics within the overall set of resumes and then numbers the inclusive topics. With the help of sun burst chart depicts individual resumes clubbed together with a unique number.

The number corresponding with the word cloud is compared with resume names. LDA topic modelling algorithm serves as an effective method for analysing the entire corpus. Since the method is unsupervised, the topics are not specified up front, and are not guaranteed to align with how a human may naturally categorize documents. The topics are learned as a probability distribution over the words that occur in each document. Each document, in turn, is described as a mixture of topics.

3.5. METHODOLOGY

3.5.1 NATURAL LANGUAGE PROCESSING

NLP, is usually distinct as the programmed automatic manipulation of natural language, such as typescript, by system.

STEP 1: WORD TOKENIZATION

Flouting the sentence into separable words is recognized as tokens. Word tokenization can tokenize the content in a manuscript whenever a space comes across and it is possible to train a model in that way. Even punctuation marks are considered personal tokens because they have certain meanings.

STEP 2: IDENTIFYING STOP WORDS

Several arguments like 'a', 'and', 'the' are repeatedly used in English. These types of arguments make a lot of clatter when doing statistical analysis. Stop Words can be

taken out by the use of NLP pipelines, which are cleansed out during statistical analysis. NLP tries to understand the dependence between different tokens in order to get the correct sense of the sentence.

STEP 3: PREDICTING PARTS OF SPEECH

Each word can be classified as an adjective, verb, adverb, noun, pronoun, etc. This will support to recognise what the sentence is chatting about. This can be accomplished by provided that a speech classification model of the pre-trained part of tokens. This model consists of several English words, encoding different parts of the speech, which categorize the similar words it may encounter in the future into different parts of the speech. Again, models do not really understand the 'sense' of words, which classifies them based on its previous experience. This is pure statistic.

STEP 4: LEMMATIZATION

The use of dependency parsing to discovery out the connection between arguments in a semantic sentence and how they re-count to each other. By the root form as the main form in the line, making a tree in the parser. It is probable to form a parsing tree of every sentence with a root word (key verb) related with it. It can recognize the thoughtful of relationship that occurs between the two arguments.

Lemmatization teaches a model by using machine learning to recognize several parts of speech, and also can teach a model to categorize the dependency among arguments by feeding many arguments. It is a tough task yet. In 2016, Google out a new dependency parser Parsey McParseface which cast-off a deep learning approach.

Lemmatization organizes the words that denote the same idea. For example: This one is the second largest town in the Belize district and are the largest in the Belize rural south consistency. 'Second', 'town' and 'largest' those tokens are represent the same meaning "Belize". To relate those words can use outcomes from dependency parsing.

3.6. TF-IDF

It is a mathematical statistic that goals to return how necessary a term is for a manuscript in a pool or corpus. It is habitually used as an evaluating aspect in examines for information recovery, text pulling out and user modelling. The TF-IDF is usually helps to correct the fact that certain words seem more regularly. TF-IDF for a term in a document is measured by multiplying two dissimilar depths.

The term frequency of an expression in a document, there are many techniques to analyse this frequency, the simplest being the original number of occurrences in which a term performs in a document. Then, there are methods to turn the frequency by the length of a document or the foundation frequency of the word that arises up in a document. If it is close to 0, a word is more common. This aspect can be occupied from the total number of documents and divided by the number of documents holding an expression, and the sum of loops can be evaluated.

Therefore, if the expression is too public and seems in multiple documents, this number will place 0. Otherwise, it will place 1. Term vectorization is a basic step in the machine learning process for analysing data, and different vectorization algorithms can severely affect the final results, so user need to choose the one that provides the results expected. Once user converted words into numbers, user can assign TF-IDF scores to tokens to make machine learning algorithms understandable, greatly improving the results of basic methods such as word counting.

A word vector refers to a document as a list of numbers, one for every possible word in the corpus. Orienting a document takes the text and creates one of these vectors, and the vectors' numbers somehow represent the content of the text.

TF-IDF gives us a way to relate every word in a document to a number that indicates how relevant each word in that document is. Then, documents with similar, relevant words will have similar vectors, which is what we are looking for in a machine learning algorithm.

TF-IDF can be useful in a number of ways. For example, TF-IDF was invented for the purpose of document search and can be used to provide the most relevant results for the user search. Imagine a task to use a search engine for searching about a celebrity. The results will be displayed in the appropriate order. Since the TF-IDF gives a high

score to the name of the celebrity, it must be said that the most relevant articles which are in the top rankings. Every search engine the user has encountered uses TF-IDF scores in its algorithm. TF-IDF is also useful for extracting keywords from the text. The highest scoring words of a document are the most relevant to that document.

3.7 COSINE SIMILARITY

This particular similarity estimates among two vectors of an expression in a document. It is evaluated by the cosine of the angle amongst two vectors and defines whether two vectors are directing in roughly the same direction. It is habitually used to evaluate document similarity in text analysis. A document can be characterized by thousands of attributes, each recording the occurrence of a certain word (such as a keyword) or phrase in the document. Thus, every document is an object characterized by term-frequency vectors. Term-frequency vectors are uncharacteristically very lengthy and sparse (i.e., they have many 0 values). Applications using such constructions contain text document clustering, information retrieval, gene feature mapping and biological taxonomy. The customary distance processes that devise studied in this chapter do not work fit for such sparse numeric data. For sample, two term frequency vectors might have many 0 values in communal, significance that the resultant documents do not share many arguments, but this does not mark them similar.

$$\sin_c(a, b) = \left(\frac{\sum_{i=1}^d a_i b_i}{\sqrt{\sum_{i=1}^d a_i^2 \cdot \sum_{i=1}^d b_i^2}} \right) \dots\dots\dots (1)$$

Need a measure that will attention on the arguments that the two documents do have in conjoint, and the incidence frequency of such arguments. In other words, we need a measurement for numeric data that pay no attention to zero matches.

Cosine similarity is a measure of similarity that can be used to relate documents or in the region of provide a ranking of documents with esteem to a given vector of inquiry arguments. Cosine Similarity Algorithm has been chosen, in which organization job +description is matched against the content of curriculum vitae in the space, and the topmost similar curriculum vitae are proposed to the recruiter.

3.8 OVERLAP COEFFICIENT SIMILARITY

The overlap coefficient is a similarity estimate that measures the overlap among two fixed collections. Overlap coefficient is alike to the jaccard directory. Overlap co-

efficient is derived as the scope of the joint of the two collections, divided by the min of the scope of the two collections,

$$O(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)} \dots\dots\dots (2)$$

If collection A is a sub-collection of B or the opposite then the overlap coefficient is equal to 1.

3.9 JACCARD SIMILARITY

Jaccard similarity coefficient also recognized as Jaccard Index, is a measurement cast-off in considerate the similarities among sample collection. This similarity can be cast-off to find the similarity among two asymmetric binary vectors collections. Index varieties from 0 to 1. The closer is 1, the more similar the two collections of data.

If two data collection exhibit the particular same numbers, their Jaccard Similarity Index will 1. Conversely, if they have no members in common then their similarity will be 0.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \dots\dots\dots (3)$$

The mass of the similarity among the finite trial collection, and is defined as the dimensions of the joint divided by the scope of the union of the trial collection.

3.10 DICE SIMILARITY

Sorensen dice index also recognized as Dice similarity co-efficient, is a mathematical tool which estimates the similarity among two collections of data.

$$2 * |X \cap Y| / (|X| + |Y|) \dots\dots\dots (4)$$

Both indices take points from 0 to 1. A rate of 1 equals that the two group of people that are relating share all their kinds, while a rate of 0 equals they share none.

CHAPTER 4

DESIGN PROCESS

4.1 SYSTEM DESIGN

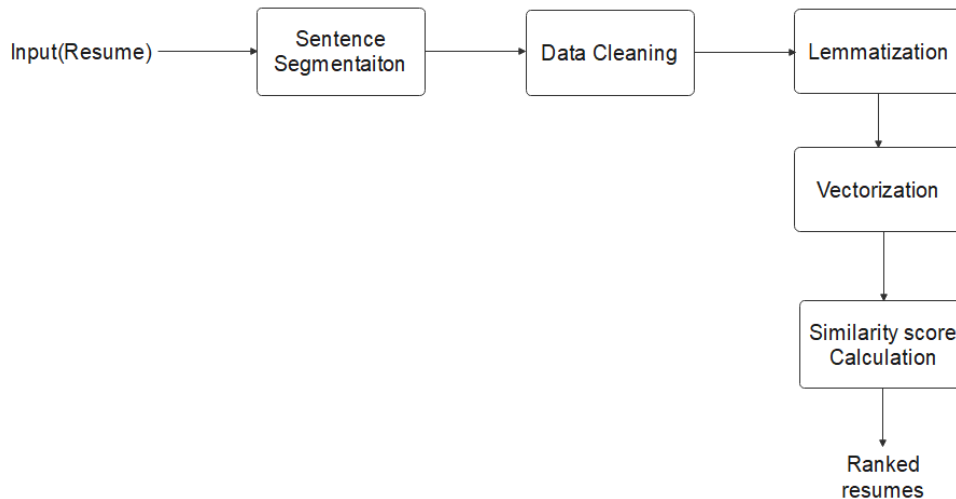


Fig. 4.1: System architecture

This system would sort use of NLP which helps in data training to make trained model and natural language processing is an analysis of natural language so that systems can recognize them. A Natural language, whether it is verbal, printed, is the most natural means of communication among individuals. It can also be the mode of expression of option for most of the documents produced. Using of natural language processing, semi structured data is transformed to a structured data with necessary extracted features.

The profiles of the new nominee are the input to the NLP by the HR team of the corporation. The module is answerable for eliminating the needless information from the curriculum vitae and provided that required data in the form of tokens which could aid in the procedure of allotment of projects to the classification module.

Resume collection is being performed and folder Structure creation is being done. Data Cleaning such as removing clutter and unnecessary punctuation would be taken off, Feature Engineering would be performed for enhancement. This contains take out punctuation, stop words, and stemming.

This library also creates a summary of the supplied information within the word limit. With the help of NLP procedures, the system which extracts the names of place, people, and other objects from text, the core area is to get a compact description of it that recollects the most significant info.

There are various different approaches of data visualization and statistical analysis procedures in the dataset that can be used to extract the data to recognise the appropriate data cleaning operations to be conducted. Also known as Exploratory data analysis, is known to be very important stage in study and inquiry of various data sets and short their significant characteristics.

The application of EDA can assist in uncovering hidden patterns in datasets and also notify how important it is in data science. EDA is often a necessary task in uncovering the hidden patterns, detecting outliers, and identifying important variables and any anomalies in data.

The data is cleaned and pre-processed at this stage, where missing and null value records are dropped. The main purpose of pre-processing is to recognize and substitute the missing values in the dataset which conquer a very small part of the entire data, to make sure an accurate result.

EDA is mainly used to realise what data can reveal beyond the formal modelling and provides an improved understanding of data set variables and the relationships between them.

It can also help determine if the statistical techniques you are considering for data analysis are appropriate. The main purpose is to support stare at data, before creating any assumption. It can help classify obvious mistakes, as well as better recognize patterns within the data, detect outliers, find remarkable relations among the variables.

The most curriculum vitae step in EDA is data cleaning that containing get rid of duplicate rows/columns, filling the void records with points like median/mean of the data, get rid of null entries, releasing various values.

4.2 SYSTEM REQUIREMENTS

Table.4.1: System requirements

HARDWARE REQUIREMENTS	SOFTWARE REQUIREMENTS
Higher RAM, of about 4GB or above	Python 3.6 and higher
Processor of frequency 1.5GHz or above	Windows and Linux

4.3 MODULES

The proposed system has the following modules,

- Latent dirichlet allocation topic modelling
- Sentence segmentation
- Data cleaning
- Lemmatization
- Vectorization
- Similarity score calculation

LATENT DIRICHLET ALLOCATION TOPIC MODELLING

LDA method is for title modelling, which classifies the text as a document and the terms for the title, which are designed based on Dirichlet distributions and processes. Latent Dirichlet Allocation has been used in the application for the following functions-

- Discovering the hidden themes in the data.
- Classifying the data into the discovered themes.
- Using the classification to organize/summarize/search the documents

Latent Dirichlet allocation is one of the most popular methods for performing topic modeling. Each document consists of various words and each topic can be associated with some words. The aim behind the LDA to find topics that the document belongs to, on the basis of words contains in it. It assumes that documents with similar topics will use a similar group of words. This enables the documents to map the probability distribution over latent topics and topics are probability distribution.

Sentence segmentation

Curriculum vitae provided as inputs will be listed in this procedure to remove any special or junk characters from the resumes. NLTK tokenizers are used to break the

database into tokens. Processes performed on tokenized databases include word removal, laminating, and vectorization.

Data cleaning

All individual characters, numbers and single characters will be removed during cleaning. Afterwards these approaches, making a clean database with no numbers, unique letters. Stop Words removal: Stop words such as like, the, was, and other words that seem repeatedly and regulator the process of decisive the prediction so that is removed.

Lemmatization:

Minimizes the phrases obtained to fully confirm whether the base word is accurately related to the language. The standard steps of lemmatization are return the corpus into a list of arguments. Generate a corpus that contains all the expression list entries that seem on the corpus. In terms of synchronization, combine expression forms with their lemmas.

Vectorization

At this stage, a dynamic Script for the TF-IDF approach is written.

$$TF - IDF(t, d) = TF(t, d) * IDF(t, d) \quad \dots\dots\dots (5)$$

$$TF(t, d) = \frac{freq(t, d)}{\sum freq(t, d)} \quad \dots\dots\dots (6)$$

$$IDF(t) = \log \left(\frac{N}{count(t)} \right) \quad \dots\dots\dots (7)$$

Where freq (t, d) is the count of the instances of the term t in document d.

TF (t, d) is the proportion of the count of term t in document d

N is the number of distinct terms in document d. It is a mathematical statistic that replicates how essential an argument into a document in a collection. TF-IDF vectorizer tokens text, learns vocabulary, converts frequency weights and encrypts new ones. It provides information on a word frequency in the documents. This algorithm issues the score to an individual argument which is computed by the use of above equations signifies more relevance in a document.

To measure the TF-IDF vector, TF-IDF Vectorizer can be used. In proposed system, designed the CV and JD as vector space. This is accomplished by compiling the

vocabulary of words found in the papers and modifying them. Every phrase matches to a vector space dimension.

Similarity score calculation

Since this is the scenario for finding document similarity, used jaccard similarity, dice similarity, overlap similarity, and cosine similarity algorithm in which the employee's job description matches the content of the resumes in space. Similar applications are recommended for the recruiter.

Beforehand measuring the cosine, jaccard, dice and overlap similarity among the job description and the Curriculum vitae's, the pre-processing algorithms cleaned curriculum vitae data and the job description into one integrated dataset.

This model is used to find applications that are closest to the specific job description. To get started, used an open source tool called gensim to calculate JD and CV. The package used provides a summary of the text given within the given words range.

The application calculates the score for the applicant's application according to the job post applied by the applicant. A ranking list will be prepared with the highest scoring applicant compared to the lowest scoring applicant according to the score received by each candidate.

By displaying an application list in chronological order related to the workplace, the technology aligns CVs with their job descriptions, making it easier for recruiters. Customized options for job descriptions are displayed in our web application. Slider options are in our application for better user experience. There will also be a bar chart showing the stats on the web application.

CHAPTER 5

IMPLEMENTATION

5.1 Drop down for displaying names

With the number of job descriptions available, Fig. 5.1 shows the names of each job description to the user. The name list of the job description should be displayed or not can be decided by the user through the drop-down box. If the option chosen is YES, then the list of job domains is shown to the user with table content. If chosen option is NO, then the job domains is not shown to the user. The job description can be selected with the help of slider module.

Should we need to Show the Job Description Names?

YES

Job No.	Job Desc. Name
0	React js developer.docx
1	Cloud specialist.docx
2	Java developer.docx
3	Cloud full stack engineer.docx

Slide to select te JD :

0

1

3

Fig.5.1: Job description name list

5.2 Drop down for displaying content

A drop down box in the GUI allows users to choose whether the content of the job description should be displayed or not. Fig. 5.2 shows job description content to the user. If the option chosen is YES, then the job description is shown to the user with the job description content. If the option chosen is NO, then the job description is not shown to the user.

Do you want Job Description to be displayed ?

YES

Job Description :

Job Description
HTML Developer Slidesigma India LLC is a web design and development company based in South Delhi. Due to the active COVID-19 crisis, we are working remotely. We are looking for an HTML developer with up to 3 years of experience. You are expected to work full-time and will be collaborating with our team and supervisors virtually. Years of Experience: 3 years experience Language: Must speak English well Duties and Requirements - Writes flawless code - Proficient with HTML, HTML5, CSS and CSS3 - Converting PSD to HTML - Other basic duties and research as required Working days and times: Monday - Saturday, 10:00 a.m. - 6:00 p.m. Contract length: 12 months Job Types: Full-time, Contract Salary: Up to 17,000.00 per month Education: Bachelor's (Required) Language: English (Preferred) Programming Languages needed: CSS (Required)

Fig.5.2: Job description content

5.3 TEXT PRE-PROCESSING

This includes removing stop words, punctuation, and stemming. This process creates a graph of sentences as the vertices. At the ends of the network there are margins that indicate how similar the two phrases are. Select the verticals with the highest scores and add them to the summary. The number of verticals to be selected is determined by the ratio or number of words.

5.4 TF-IDF CALCULATION

Produce the info on the occurrence of a token in documents. TF-IDF score of a word calculated using the equation specifies greater significance in a document. In proposed system, designed the CV and JD as vector space. This is accomplished by compiling the vocabulary of words found in the papers and modifying them. Every phrase matches to a vector space dimension.

5.5 RANKING EACH CANDIDATES THROUGH SCORES

In this approach, the job description of the employer matches the content of the applications in the space, and the best n compatibility applications are recommended to the employer. Before calculating the jaccard similarity, Dice similarity, Cosine Similarity, Overlap similarity between JD and the resume, the model combines the refined resume data and the job description into a single data set.

Jaccard, Dice, Cosine, Overlap similarity is calculated among each curriculum vitae and job descriptions specified in the 'n' vector space using the equation. According to the similarity scores, the applicant's will receive a ranked. Centred on the ranks allotted, resumes with the first n rank is recommended to the job recruiters. Score displayed to the user according to the score table.

Table. 5.1: Score Table

Rank	Name	Scores
1	M .Poonkodi.docx	84.55547782550926
2	S.Sridhar.docx	84.25829801171213
3	M.Indumathy.docx	84.25462865416111
4	N. Muthurasu.docx	84.04389303773878
5	T. Malathi.docx	84.04242711561236
6	C. Sabarinathan.docx	82.98761325282227
7	G. Merlin Linda.docx	82.38512491233925
8	M.Ramesh.docx	82.22058281337696
9	J. Arun nehru.docx	80.73975026803352
10	G. PaavaiAnand.docx	80.70649329556217

The bar chart (Fig.5.3) shows the result of score table by plotting chart with appropriate colours for all candidates applying for the job. The colours depict all the users based on the score calculated by the similarities and also indicates which applicant has different colours within the scope of score calculation

Score and Rank Distribution

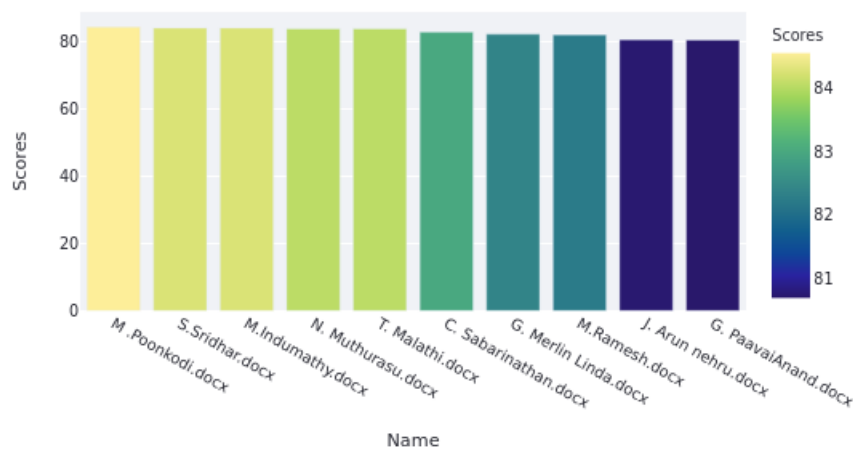


Fig. 5.3 Bar chart

Using the LDA algorithm, through unsupervised classification methodology called Topic Modeling is implemented. With automated analysis of each resumes, utilizing Fig.5.4 helps to highlight each topics within the overall set of resumes and then numbers the inclusive topics.

Topics and Topic Related Keywords

This Wordcloud representation shows the Topic Number and the Top Keywords that constitute a Topic. This further is used to cluster the resumes.

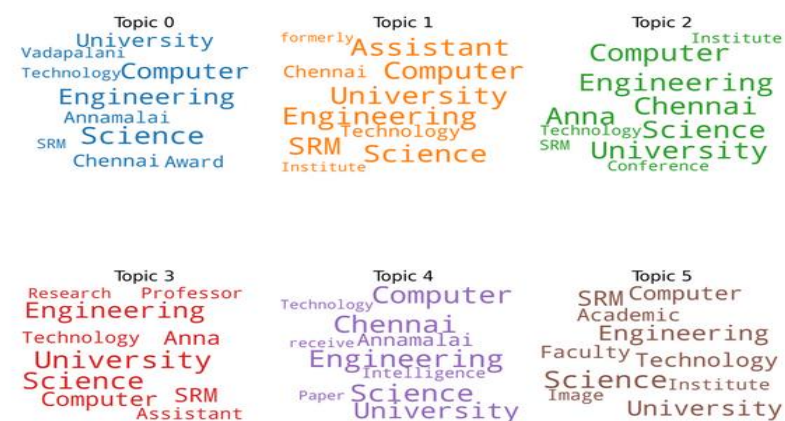


Fig.5.4: Word cloud

Fig.5.5 depicts individual resumes clubbed together with a unique number. The number corresponding with the word cloud is compared with resume names. With the word cloud present, it is possible

Topic Modeling with Resume using LDA

Using LDA to divide the topics into a number of usefull topics and creating a Cluster of matching topic resumes.

Topic Distribution Graph

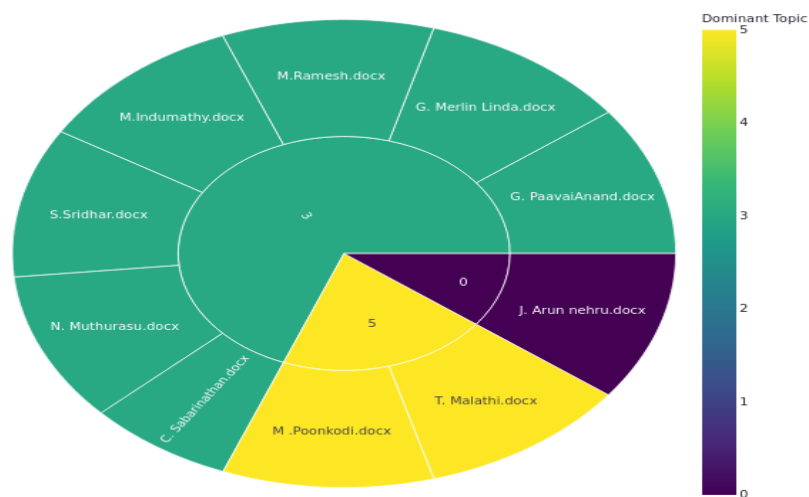


Fig. 5.5: Topic modelling

CHAPTER 6

6. CONCLUSION AND FUTURE ENHANCEMENT

The company receives adequate number of job applicant's resumes for each job description. Discovering the correct candidate from the application seeing a bio-data is an energy to spend time with any corporation now-a-days. Categorizing a candidate's application by physically is a difficult and time-consuming and resource-intensive process. To solve this difficulties, proposed system is generated for an automated machine learning based system that recommends the applicant applications to HR that can be accepted based on the job description provided. The recommended method consisted of two stages: First, it classified the bio-data into different groups. Second, it recommends applications based on similarity with the job description. Using NLP and Jaccard, Dice, Cosine, Overlap similarity to rank applications allowing to a given restraint, this intelligence system ranks a resume of any design according to the given restrictions or the following requirements issued by the client company.

Proposed system will take the set of job applicant's resumes as an input from the client corporation and that Client corporation will provide the job description and the application will be rated by the organization accordingly. Future Enhancements is to focus on ranking applications for people in different languages such as German, French or Hindi, through exploring additional ways to analyse and implement the model in major languages of the world.

REFERENCES:

- [1] V. V. Dixit, Trisha Patel, Nidhi Deshpande and Kamini Sonawane, "Resume Sorting using Artificial Intelligence", in IJERT, Paper ID : IJERTV10IS090197, vol.10, Issue 09, September-2021, <https://www.ijert.org/personality-prediction-via-cv-analysis-using-machine-learning>
- [2] Md Tanzim Reza, and Md. Sakib Zaman, "Analyzing CV/Resume using natural language processing and machine learning", BRAC University, 2017, <http://hdl.handle.net/10361/9480>
- [3] Sujit Amin, Nikita Jayakar, Sonia Sunny, Pheba Babu, M. Kiruthika and Ambarish Gurjar, "Web Application for Screening Resume", in International Conference on Nascent Technologies in Engineering (ICNTE), January 2020, <https://ieeexplore.ieee.org/document/8945869/authors#authors>
- [4] Jitendra Purohit, Aditya Bagwe, Rishabh Mehta, Ojaswini Mangaonka and Elizabeth George, "Natural Language Processing based Jaro-The Interviewing Chatbot", in 3rd International Conference on Computing Methodologies and Communication (ICCMC), August 2019, <https://ieeexplore.ieee.org/document/8819708/authors#authors>
- [5] Tejaswini K, Umadevi V, Shashank M Kadiwal, Sanjay Revanna, "Design and Development of Machine Learning based Resume Ranking System", in Global Transitions Proceedings, vol. 2, issue 2, October 2021, <https://www.sciencedirect.com/science/article/pii/S2666285X21001011>
- [6] Pradeep Kumar Roy, Sarabjeet Singh Chowdhary and Rocky Bhatia, "A Machine Learning approach for automation of Resume Recommendation system", in Procedia Computer Science, vol. 167, Pp 2318-2327, 2020, <https://www.sciencedirect.com/science/article/pii/S187705092030750X>
- [7] Abeer Zaroor, Mohammed Maree, and Muath Sabha, "A Hybrid Approach to Conceptual Classification and Ranking of Resumes and Their Corresponding Job Post", Smart Innovation, Systems and Technologies book series, vol. 72, 2017, https://link.springer.com/chapter/10.1007/978-3-319-59421-7_10
- [8] Irfan Ali, Nimra Mughal, Zahid Hussain Khand, Javed Ahmed and Ghulam Mujtaba, "Resume Classification System using Natural Language Processing and Machine Learning Techniques," in Mehran University Research Journal of Engineering and Technology, Vol. 41, No. 1, January 2022 [p-ISSN: 0254-7821, e-ISSN: 2413-7219], <https://publications.muet.edu.pk/index.php/muetri/article/view/2353>

- [9] Anushka Sharma, Smiti Singhal and Dhara Ajudia, "Intelligent Recruitment System Using NLP", in International Conference on Artificial Intelligence and Machine Vision (AIMV), pp 1-5, 2021, <https://ieeexplore.ieee.org/document/9670958>
- [10] Farber, F. Weitzel, T., Keim. T. 2003, "An automated recommendation approach to selection in personnel recruitment" AMCIS 2003 proceedings, <https://aisel.aisnet.org/cgi/viewcontent.cgi?article=1768&context=amcis2003>
- [11] Atharva Kulkarni, Tanuj Shankarwar, and Siddharth Thorat, "Personality Prediction Via CV Analysis using Machine Learning ", in International Journal of Research in Engineering, Science and Management vol -2, Issue-4, ISSN (Online): 2581-5792, April-2019, https://www.ijresm.com/Vol.2_2019/Vol2_Iss4_April19/IJRESM_V2_I4_117.pdf
- [12] Ankit Basarkar, "Document Classification Using Machine Learning" in SJSU ScholarWorks Spring, may-2017, https://scholarworks.sjsu.edu/etd_projects/531/
- [13] Vishnu M Menon and H A Rahulnath "A novel approach to evaluate and rank candidates in a recruitment process by estimating emotional intelligence through social media data" in 2016 International Conference on Next Generation Intelligent Systems (ICNGIS), vol. 1-6, sept 2016), <https://ieeexplore.ieee.org/document/7854061>
- [14] Rohini Nimbekar, Yaqesh Patil, Rahul Prabhu and Shainila Mulla, "Automated Resume Evaluation System using NLP", in 2019 International Conference on Advances in Computing, Communication and Control (ICAC3), pp. 1-4, 2019, <https://ieeexplore.ieee.org/abstract/document/9036842>
- [15] Allan Robey, Kaushik Shukla, Kashish Agarwal, Keval Joshi, Professor Shalmali Joshi, "Personality Prediction System through CV Analysis", in International Research Journal of Engineering and Technology (IRJET) e-ISSN: 2395-0056 Volume: 06 Issue: 02, Feb 2019, <https://www.ijert.org/personality-prediction-via-cv-analysis-using-machine-learning>

Appendix

A1-Source code

app.py

```
import matplotlib.colors as mcolors

import gensim

import gensim.corpora as corpora

from operator import index

from wordcloud import WordCloud

from pandas._config.config import options

import pandas as pd

import streamlit as st

import plotly.express as px

import plotly.graph_objects as go

import matplotlib.pyplot as plt

import Similar

from PIL import Image


st.title("Automated Resume Shortlisting System")


image = Image.open('Images//logo.png')

st.image(image, use_column_width=True)


Resumes = pd.read_csv('Resume_Data.csv')

Jobs = pd.read_csv('Job_Data.csv')
```

```

if len(Jobs['Name']) == 1:

    st.write(

        "There is only 1 Job Description present. It will be used to create scores.")

else:

    st.write("There are ", len(Jobs['Name']),

        "Job Descriptions available. Please select one.")


if len(Jobs['Name']) >= 1:

    option_yn = st.selectbox(

        "Should we need to Show the Job Description Names?", options=['YES', 'NO'])

    if option_yn == 'YES':

        index = [a for a in range(len(Jobs['Name']))]

        fig = go.Figure(data=[go.Table(header=dict(values=["Job No.", "Job Desc.
Name"]), line_color='darkslategray',

            fill_color='lightskyblue'),

            cells=dict(values=[index, Jobs['Name']], line_color='darkslategray',

                fill_color='cyan'))

        ])

        fig.update_layout(width=700, height=400)

        st.write(fig)


index = st.slider("Slide to select te JD : ", 0,

    len(Jobs['Name'])-1, 1)

```



```
option_yn = st.selectbox("Do you want Job Description to be displayed ?",
options=['YES', 'NO'])
```

```
if option_yn == 'YES':
```

```
    st.markdown("---")
```

```
    st.markdown("### Job Description :")
```

```
    fig = go.Figure(data=[go.Table(
        header=dict(values=["Job Description"],
            fill_color='#f0a500',
            align='center', font=dict(color='white', size=16)),
        cells=dict(values=[Jobs['Context'][index]],
            fill_color='#f4f4f4',
            align='left'))])
```

```
    fig.update_layout(width=800, height=500)
```

```
    st.write(fig)
```

```
    st.markdown("---")
```

```
@st.cache()
```

```
def calculate_scores(resumes, job_description):
```

```
    scores = []
```

```
    for x in range(resumes.shape[0]):
```

```
        score = Similar.match(
            resumes['TF_Based'][x], job_description['TF_Based'][index])
        scores.append(score)
```

```
    return scores
```

```
Resumes['Scores'] = calculate_scores(Resumes, Jobs)
```

```
Ranked_resumes = Resumes.sort_values(  
    by=['Scores'], ascending=False).reset_index(drop=True)
```

```
Ranked_resumes['Rank'] = pd.DataFrame(  
    [i for i in range(1, len(Ranked_resumes['Scores'])+1)])
```

```
fig1 = go.Figure(data=[go.Table(  
    header=dict(values=["Rank", "Name", "Scores"],  
        fill_color='#00416d',  
        align='center', font=dict(color='white', size=16)),  
    cells=dict(values=[Ranked_resumes.Rank, Ranked_resumes.Name,  
Ranked_resumes.Scores],  
        fill_color='#d6e0f0',  
        align='left'))])
```

```
fig1.update_layout(title="Top Ranked Resumes", width=700, height=1100)
```

```
st.write(fig1)
```

```
st.markdown("---")
```

```
fig2 = px.bar(Ranked_resumes,  
    x=Ranked_resumes['Name'], y=Ranked_resumes['Scores'], color='Scores',
```

```

        color_continuous_scale='haline', title="Score and Rank Distribution")

#fig2.update_layout(width=700, height=700)

st.write(fig2)


st.markdown("---")


@st.cache()
def get_list_of_words(document):
    Document = []

    for a in document:
        raw = a.split(" ")
        Document.append(raw)

    return Document

document = get_list_of_words(Resumes['Cleaned'])

id2word = corpora.Dictionary(document)

corpus = [id2word.doc2bow(text) for text in document]

lda_model = gensim.models.LdaModel(corpus=corpus, id2word=id2word,
num_topics=6, random_state=100,
                                update_every=3, chunksize=100, passes=50, alpha='auto',
per_word_topics=True)

```

```
@st.cache
```

```
def format_topics_sentences(ldamodel, corpus):
```

```
    sent_topics_df = []
```

```
    for i, row_list in enumerate(ldamodel[corpus]):
```

```
        row = row_list[0] if ldamodel.per_word_topics else row_list
```

```
        row = sorted(row, key=lambda x: (x[1]), reverse=True)
```

```
        for j, (topic_num, prop_topic) in enumerate(row):
```

```
            if j == 0:
```

```
                wp = ldamodel.show_topic(topic_num)
```

```
                topic_keywords = ", ".join([word for word, prop in wp])
```

```
                sent_topics_df.append(
```

```
                    [i, int(topic_num), round(prop_topic, 4)*100, topic_keywords])
```

```
            else:
```

```
                break
```

```
    return sent_topics_df
```

```
st.markdown("### Topics and Topic Related Keywords ")
```

```
st.markdown(
```

```
    """This Wordcloud representation shows the Topic Number and the Top Keywords  
    that constitute a Topic.
```

```
    This further is used to cluster the resumes.    """)
```

```
cols = [color for name, color in mcolors.TABLEAU_COLORS.items()]
```

```

cloud = WordCloud(background_color='white',
                  width=2500,
                  height=1800,
                  max_words=10,
                  colormap='tab10',
                  collocations=False,
                  color_func=lambda *args, **kwargs: cols[i],
                  prefer_horizontal=1.0)

```

```

topics = lda_model.show_topics(formatted=False)

```

```

fig, axes = plt.subplots(2, 3, figsize=(10, 10), sharex=True, sharey=True)

```

```

for i, ax in enumerate(axes.flatten()):

```

```

    fig.add_subplot(ax)

```

```

    topic_words = dict(topics[i][1])

```

```

    cloud.generate_from_frequencies(topic_words, max_font_size=300)

```

```

    plt.gca().imshow(cloud)

```

```

    plt.gca().set_title('Topic ' + str(i), fontdict=dict(size=16))

```

```

    plt.gca().axis('off')

```

```

plt.subplots_adjust(wspace=0, hspace=0)

```

```

plt.axis('off')

```

```
plt.margins(x=0, y=0)
```

```
plt.tight_layout()
```

```
st.pyplot(plt)
```

```
st.markdown("---")
```

```
df_topic_sents_keywords = format_topics_sentences(
```

```
    ldamodel=lda_model, corpus=corpus)
```

```
df_some = pd.DataFrame(df_topic_sents_keywords, columns=[
```

```
    'Document No', 'Dominant Topic', 'Topic % Contribution', 'Keywords'])
```

```
df_some['Names'] = Resumes['Name']
```

```
df = df_some
```

```
st.markdown("## Topic Modeling with Resume using LDA ")
```

```
st.markdown(
```

```
    "Using LDA to divide the topics into a number of usefull topics and creating a Cluster  
of matching topic resumes. ")
```

```
fig3 = px.sunburst(df, path=['Dominant Topic', 'Names'], values='Topic %  
Contribution',
```

```
    color='Dominant Topic', color_continuous_scale='viridis', width=800,  
height=800, title="Topic Distribution Graph")
```

```
st.write(fig3)
```

Cleanner.py

```
import spacy
```

```
import Distill
```

```
try:
```

```
    nlp = spacy.load('en_core_web_sm')
```

```
except ImportError:
```

```
    print("Spacy's English Language Modules aren't present \n Install them by doing \n  
python -m spacy download en_core_web_sm")
```

```
def _base_clean(text):
```

```
    text = Distill.tokenize(text)
```

```
    text = Distill.remove_stopwords(text)
```

```
    text = Distill.remove_tags(text)
```

```
    text = Distill.lemmatize(text)
```

```
    return text
```

```
def _reduce_redundancy(text):
```

```
    return list(set(text))
```

```
def _get_target_words(text):
```

```
    target = []
```

```
    sent = " ".join(text)
```

```
    doc = nlp(sent)
```

```
for token in doc:

    if token.tag_ in ['NN', 'NNP']:

        target.append(token.text)

return target
```

```
def Cleaner(text):

    sentence = []

    sentence_cleaned = _base_clean(text)

    sentence.append(sentence_cleaned)

    sentence_reduced = _reduce_redundancy(sentence_cleaned)

    sentence.append(sentence_reduced)

    sentence_targetted = _get_target_words(sentence_reduced)

    sentence.append(sentence_targetted)

    return sentence
```

Distill.py

```
import spacy

import re

from nltk.tokenize import word_tokenize

from nltk.corpus import stopwords
```



```
stop_words = stopwords.words('english')
```

```
nlp = spacy.load('en_core_web_sm')
```

```
def remove_stopwords(text, stopwords=stop_words, optional_params=False,  
optional_words=[]):
```

```
    if optional_params:
```

```
        stopwords.append([a for a in optional_words])
```

```
    return [word for word in text if word not in stopwords]
```

```
def tokenize(text):
```

```
    text = re.sub(r'^\w\s', "", text)
```

```
    return word_tokenize(text)
```

```
def lemmatize(text):
```

```
    str_text = nlp(" ".join(text))
```

```
    lemmatized_text = []
```

```
    for word in str_text:
```

```
        lemmatized_text.append(word.lemma_)
```

```
    return lemmatized_text
```

```
def remove_tags(text, postags=['PROPN', 'NOUN', 'ADJ', 'VERB', 'ADV']):
```

```
    filtered = []
```

```

str_text = nlp(" ".join(text))

for token in str_text:

    if token.pos_ in postags:

        filtered.append(token.text)

return filtered

```

FileReader.py

```

import Cleaner

import textract as tx

import pandas as pd

import os

import tf_idf


resume_dir = "Data/Resumes/"

job_desc_dir = "Data/JobDesc/"

resume_names = os.listdir(resume_dir)

job_description_names = os.listdir(job_desc_dir)


document = []


def read_resumes(list_of_resumes, resume_directory):

    placeholder = []

    for res in list_of_resumes:

```

```

temp = []

temp.append(res)

text = tx.process(resume_directory+res, encoding='ascii')

text = str(text, 'utf-8')

temp.append(text)

placeholder.append(temp)

return placeholder


document = read_resumes(resume_names, resume_dir)


def get_cleaned_words(document):

    for i in range(len(document)):

        raw = Cleaner.Cleaner(document[i][1])

        document[i].append(" ".join(raw[0]))

        document[i].append(" ".join(raw[1]))

        document[i].append(" ".join(raw[2]))

        sentence = tf_idf.do_tfidf(document[i][3].split(" "))

        document[i].append(sentence)

    return document


Doc = get_cleaned_words(document)


Database = pd.DataFrame(document, columns=[

    "Name", "Context", "Cleaned", "Selective", "Selective_Reduced",

    "TF_Based"])

```

```
Database.to_csv("Resume_Data.csv", index=False)
```

```
def read_jobdescriptions(job_description_names, job_desc_dir):
```

```
    placeholder = []
```

```
    for tes in job_description_names:
```

```
        temp = []
```

```
        temp.append(tes)
```

```
        text = tx.process(job_desc_dir+tes, encoding='ascii')
```

```
        text = str(text, 'utf-8')
```

```
        temp.append(text)
```

```
        placeholder.append(temp)
```

```
    return placeholder
```

```
job_document = read_jobdescriptions(job_description_names, job_desc_dir)
```

```
Jd = get_cleaned_words(job_document)
```

```
jd_database = pd.DataFrame(Jd, columns=[
```

```
    "Name", "Context", "Cleaned", "Selective", "Selective_Reduced",
```

```
    "TF_Based"])
```

```
jd_database.to_csv("Job_Data.csv", index=False)
```

Similar.py

```
import textdistance as td
```

```
def match(resume, job_des):
```

```
    j = td.jaccard.similarity(resume, job_des)
```

```
    s = td.sorensen_dice.similarity(resume, job_des)
```

```
    c = td.cosine.similarity(resume, job_des)
```

```
    o = td.overlap.normalized_similarity(resume, job_des)
```

```
    total = (j+s+c+o)/4
```

```
    return total*100
```

tf_idf.py

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
def do_tfidf(token):
```

```
    tfidf = TfidfVectorizer(max_df=0.05, min_df=0.002)
```

```
    words = tfidf.fit_transform(token)
```

```
    sentence = " ".join(tfidf.get_feature_names())
```

```
    return sentence
```

A2-Screenshots

Automated Resume Shortlisting System



There are 4 Job Descriptions available. Please select one.

Fig. 8.1: No. of. job description

Should we need to Show the Job Description Names?

YES

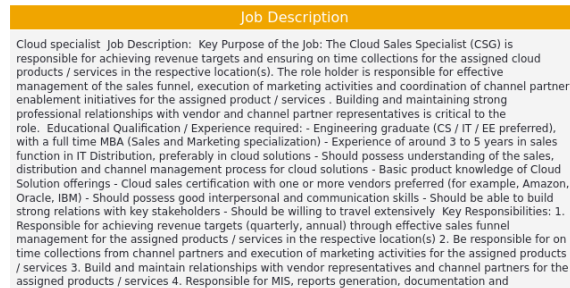
Job No.	Job Desc. Name
0	React js developer.docx
1	Cloud specialist.docx
2	Java developer.docx
3	Cloud full stack engineer.docx

Slide to select te JD :



Fig. 8.2: Job description selection

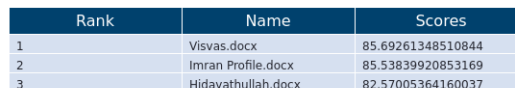
Job Description :



Job Description
Cloud specialist Job Description: Key Purpose of the Job: The Cloud Sales Specialist (CSG) is responsible for achieving revenue targets and ensuring on time collections for the assigned cloud products / services in the respective location(s). The role holder is responsible for effective management of the sales funnel, execution of marketing activities and coordination of channel partner enablement initiatives for the assigned product / services . Building and maintaining strong professional relationships with vendor and channel partner representatives is critical to the role. Educational Qualification / Experience required: - Engineering graduate (CS / IT / EE preferred), with a full time MBA (Sales and Marketing specialization) - Experience of around 3 to 5 years in sales function in IT Distribution, preferably in cloud solutions - Should possess understanding of the sales, distribution and channel management process for cloud solutions - Basic product knowledge of Cloud Solution offerings - Cloud sales certification with one or more vendors preferred (for example, Amazon, Oracle, IBM) - Should possess good interpersonal and communication skills - Should be able to build strong relations with key stakeholders - Should be willing to travel extensively Key Responsibilities: 1. Responsible for achieving revenue targets (quarterly, annual) through effective sales funnel management for the assigned products / services in the respective location(s) 2. Be responsible for on time collections from channel partners and execution of marketing activities for the assigned products / services 3. Build and maintain relationships with vendor representatives and channel partners for the assigned products / services 4. Responsible for MIS, reports generation, documentation and

Fig. 8.3: Job description content

Top Ranked Resumes



Rank	Name	Scores
1	Visvas.docx	85.69261348510844
2	Imran Profile.docx	85.53839920853169
3	Hidayathullah.docx	82.57005364160037

Fig. 8.4: Ranking table

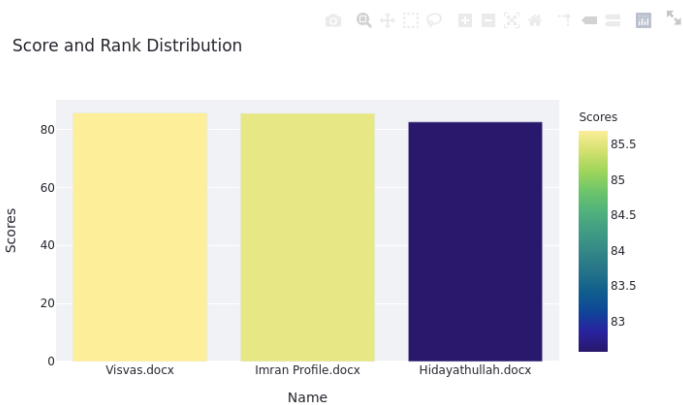


Fig. 8.5: Bar chart for ranking

Topics and Topic Related Keywords

This Wordcloud representation shows the Topic Number and the Top Keywords that constitute a Topic. This further is used to cluster the resumes.

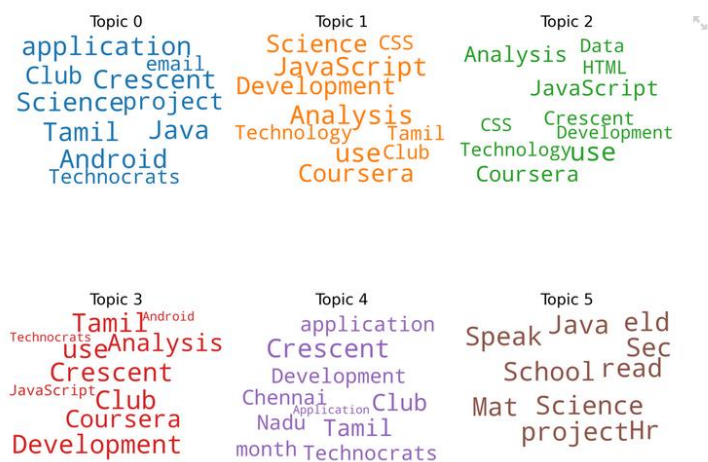


Fig. 8.6: Word Cloud

Topic Modeling with Resume using LDA

Using LDA to divide the topics into a number of useful topics and creating a Cluster of matching topic resumes.

Topic Distribution Graph

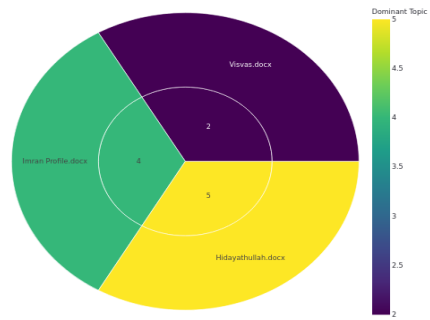


Fig. 8.7: Topic Modelling for individual

Project report 2

by Vijayalakshmi C

Submission date: 07-Jun-2022 03:05PM (UTC+0530)

Submission ID: 1852184108

File name: Batch_18_to_33.docx (784.61K)

Word count: 124416

Character count: 674423

TECHNICAL BIOGRAPHY



Mr. David (180071601072) is a highly self motivated, goal oriented and a focused engineer with excellent communication skills. He is currently pursuing his undergraduate program as a computer science graduate at B. S. Abdur Rahman Crescent Institute Of Science And Technology. Apart from his regular studies, he has a strong interest at artificial intelligence and machine learning, and also cloud computing. He has a intermediate knowledge on OOPS language such as C++ and python. He also has a hand on web development technologies with a strong coding skills with the languages such as HTML, CSS and JS.



Mr. Mohamed Imran (180071601108) completed 10th in 2016 from mount park higher secondary school thiyagathurgam with securing marks 80.4%.I have a completed 12th in 2018 from mount park park higher secondary school thiyagathurgam with securing marks 80.3%. And currently pursuing Bachelor of technology in computer science engineering form B. S. Abdur rahman crescent Institute of science and Technology vandalur with securing 83.4% till 6th semester. As per additional course, completed Android application development course and have an intermediate knowledge of C++, java, HTML and Python.